

Guia Completo para Memorizar Código e Conceitos

ÍNDICE

1. [Pinia Store](#)
 2. [Vue Template Directives](#)
 3. [Vue Reatividade](#)
 4. [Vue Lifecycle](#)
 5. [Vue Components](#)
 6. [Fetch API](#)
 7. [CSS Layouts](#)
 8. [Princípios de Usabilidade](#)
 9. [Heurísticas de Nielsen](#)
 10. [Métodos de Avaliação](#)
 11. [WCAG](#)
 12. [Modelo de Norman](#)
 13. [Acessibilidade](#)
 14. [JavaScript](#)
-

1. PINIA STORE

Mnemónica Principal: "SGA - Sistema de Gestão Automático"

S = STATE (dados reativos)
G = GETTERS (valores calculados)
A = ACTIONS (modificar estado)

 **Analogia: A Loja de Roupas**

javascript

// PINIA = LOJA DE ROUPA

```
state: () => ({
  // 💰 STATE = INVENTÁRIO da loja (o que tens)
  items: [],
  money: 0
})

getters: {
  // 🪞 GETTERS = VITRINE (mostra info sobre inventário)
  // Só OLHAS, não MEXES!
  totalItems: (state) => state.items.length,
  expensiveItems: (state) => state.items.filter(i => i.price > 100)
}

actions: {
  // 🛠️ ACTIONS = FUNCIONÁRIOS (fazem mudanças)
  // Podem MODIFICAR o inventário
  addItem(item) {
    this.items.push(item); // MEXE no estado
  }
}
```

Regras de Ouro Pinia:

- ✓ STATE = função () => ({}): "State Sempre função"
- ✓ GETTERS = (state) => valor: "Getters Geram valores"
- ✓ ACTIONS = this.propriedade: "Actions Agem com this"

- ✗ data: function() → NÃO! "Data é de Vue components"
- ✗ methods: {} → NÃO! "Methods também são de components"
- ✗ getters modificam → NÃO! "Getters Garantem sem modificar"

Musiquinha Pinia:

"STATE guarda, GETTERS calcula,
ACTIONS muda - nunca te enganas!
data e methods? Isso é pra components,
Na store é SGA - são os componentes!"

2. VUE TEMPLATE DIRECTIVES

Mnemónica: "v-MIFSE-BC" (pronuncia: "vê-mifse-bê-cê")

v-Model = two-way binding (📝 escreve E lê)
v-If = existe/não existe (🗑️ adiciona/remove DOM)
v-For = loop/repetição (🔄 lista)
v-Show = mostrar/esconder (👁️ CSS display)
v-Else = senão (só com v-if!)

v-Bind = :prop (🔗 liga dados)
v-Click = @click (👉 eventos)

🧠 Mapa Mental Template:

```

vue

<template>
  <!-- 📝 MODEL = Papel e caneta (escreves E lê) -->
  <input v-model="text" />

  <!-- 🗑️ IF = Palco (elemento EXISTE ou NÃO) -->
  <div v-if="show">Estou no palco!</div>

  <!-- 📋 FOR = Fotocópias (repetir elementos) -->
  <li v-for="item in items" :key="item.id">

  <!-- 👁️ SHOW = Cortina (existe sempre, só esconde) -->
  <div v-show="visible">Atrás da cortina!</div>

  <!-- 🔗 BIND = Corda (:prop liga dados) -->
  

  <!-- 👉 CLICK = Botão (@evento) -->
  <button @click="action">Click!</button>
</template>
  
```

📊 Tabela Rápida:

Diretiva	Atalho	O que faz	Quando
v-model	-	⇄ two-way	formulários
v-bind	:	→ one-way	props/attrs
v-on	@	← eventos	interação
v-if	-	🗑️ remove DOM	condicional
v-show	-	👁️ display:none	toggle +++
v-for	-	🔄 loop	listas

🎯 Regra do :key em v-for:

"Para FORMar lista precisa KEY,
ID único é a melhor way,
Index só se nunca mudar,
Senão Vue vai se atrapalhar!"

✓ :key="item.id" "ID é Ideal"
✓ :key="item.name" "Name também dá"
✗ :key="index" "Index? Nããão!" (se lista muda)

💡 v-if vs v-show:

v-IF = "I'm Fully gone" (remove completamente)
v-SHOW = "Still Here, Only" (só esconde, continua no DOM)

Usa v-show quando: toggle FREQUENTE
Usa v-if quando: raramente muda OU condição complexa

3. VUE REATIVIDADE ⚡

Mnemónica: "RRC" - Ref, Reactive, Computed

R = REF → pRimitivos (números, strings, booleans)
R = REACTIVE → objeRtos e aRRays
C = COMPUTED → Calculados (derivados)

🎨 Analogia Visual:

```
javascript

// 📦 REF = Caixa com 1 item (primitivo)
const count = ref(0);
// Para VER/MUDAR: abrir a .value (no script)
count.value++;

// 📦 REACTIVE = Caixote com vários items (objeto)
const user = reactive({ name: 'João', age: 25 });
// Para VER/MUDAR: acesso direto (sem .value)
user.name = 'Maria';

// 🧮 COMPUTED = Calculadora automática
const doubled = computed(() => count.value * 2);
// Só LÊ! Recalcula sozinha quando count muda
```

📝 Regra de Ouro:

"REF para Números e Texto,
REACTIVE para Objeto completo,
COMPUTED calcula perfeito!"

ou

"Primitivo? REF sem medo!
Objeto? REACTIVE segredo!
Calcular? COMPUTED é o remédio!"

Cheat Mental:

ref(0) → número → .value no script ✓
ref('text') → string → .value no script ✓
ref(true) → boolean → .value no script ✓
reactive(0) → **✗ ERRO!** → usar ref() !
reactive({}) → objeto → SEM .value ✓
reactive([]) → array → SEM .value ✓

Template: {{ count }} → SEMPRE sem .value
Script: count.value++ → COM .value (só ref)

Watch vs WatchEffect:

watch = "Vou Assistir Tarde"
→ NÃO executa imediatamente (precisa immediate: true)
→ Vês mudanças: (newVal, oldVal) => {}

watchEffect = "Watch Executa TUDO!"
→ Executa IMEDIATAMENTE
→ Tracking automático
→ Só newVal

4. VUE LIFECYCLE

Mnemónica: "BBC MUM" (British Broadcasting Corporation MUM)

B = Before Mount → antes de aparecer no ecrã
B = Before Update → antes de atualizar
C = Created → criado (Options API)

M = Mounted → apareceu no ecrã ✨ (FETCH AQUI!)
U = Updated → atualizou
M = unMounted → removido (CLEANUP AQUI! 🧹)

Analogia: Filme no Cinema

javascript

```
// 🎬 BEFORE MOUNT = Preparar sala antes do filme
onBeforeMount() => {
  console.log('Preparando pipocas...');
};

// 🎭 MOUNTED = Filme começou! (luzes apagam)
// 💡 MOMENTO IDEAL PARA FETCH!
onMounted() => {
  fetchData(); // ✓ Buscar dados
  startInterval(); // ✓ Iniciar timers
  setupEventListeners(); // ✓ Adicionar listeners
};

// 🔄 UPDATED = Nova cena do filme
onUpdated() => {
  console.log('DOM atualizou');
};

// 🧹 UNMOUNTED = Filme acabou, limpar sala
// ⚠️ ESSENCIAL FAZER CLEANUP!!!
onUnmounted() => {
  clearInterval(intervalId); // ✓ Limpar timers
  removeEventListeners(); // ✓ Remover listeners
  // SEM ISTO = MEMORY LEAK! 💣
};
```

🎯 Regra Crítica:

"MOUNTED faz FETCH,
UNMOUNTED faz CLEANUP,
Esquecer? Sistema vai CRASHUP!"

ou

"Se MOUNTED iniciou,
UNMOUNTED tem que limpar,
Senão o código ficou a executar!"

📊 Ordem dos Lifecycle Hooks:

1. setup() / created → 🏗️ construção
2. onBeforeMount → 🎬 antes de aparecer
3. onMounted ★ FETCH! → ✨ apareceu (DOM pronto)
4. onBeforeUpdate → 🔄 vai atualizar
5. onUpdated → ✅ atualizou
6. onBeforeUnmount → 🚶 vai sair
7. onUnmounted ★ CLEANUP! → 🧹 saiu (limpar!)

5. VUE COMPONENTS 🧩

Mnemónica: "PES" - Props, Emits, Slots

P = PROPS → PAI dá ao filho (↓)
E = EMITS → filho Envia ao pai (↑)
S = SLOTS → conteúdo passa pelo buraco (●)

🍷 Analogia: Correios

javascript

```
// 📧 PROPS = Carta que RECEBES (read-only!)
const props = defineProps(['message']);
// ✗ props.message = 'novo' → NÃO PODES MODIFICAR!

// 📬 EMITS = Carta que ENVIAS
const emit = defineEmits(['update']);
emit('update', 'novo valor'); // ✓ envia para pai

// 🕒 SLOTS = Buraco na parede (passa conteúdo)
<slot name="header">Default</slot>
```

📝 Regras de Ouro:

"PROPS são Presentes (só recibes, não alteras)
EMITS são E-mails (envias mensagens)
SLOTS são eSpaços (para meter conteúdo)"

Props: ↓ Descem (pai → filho)
Emits: ↑ Sobem (filho → pai)
Slots: ↔ Passam (conteúdo através)

🎯 v-model em Components:

v-model é ATALHO para:
:modelValue + @update:modelValue

"Model vai e volta,
Value desce e update sobe!"

```
// Filho:
<input
  :value="modelValue"
  @input="$emit('update:modelValue', $event.target.value)"
/>
```

6. FETCH API 🌐

Mnemónica: "FPJ-TCD" (Fetch, Promise, JSON - Then, Catch, Data)

F = Fetch → inicia pedido
P = Promise → retorna promessa
J = JSON → também é Promise!
-
T = Then → quando resolve
C = Catch → quando erro
D = Data → finalmente os dados!

🍕 Analogia: Pedir Pizza

javascript

```
// 1. ☎️ FETCH = Ligar para pizzeria
fetch('/api/pizza')

// 2. 📦 .then(response) = Receber caixa fechada
.then(response => response.json())
// ⚠️ Ainda não é a pizza! Precisa abrir caixa!

// 3. 🍕 .then(data) = Pizza dentro da caixa
.then(pizza => {
  this.myPizza = pizza; // ✅ Agora sim!
})

// 4. 😞 .catch = Se deu erro
.catch(error => {
  console.log('Pizza não chegou!');
});
```

⚠️ ERRO CLÁSSICO:

javascript

```
// ❌ ERRADO - Guardas a CAIXA, não a PIZZA!
this.pizza = fetch('/api/pizza');
// Isto guarda Promise, não os dados!

// ❌ ERRADO - Guardas a caixa FECHADA!
this.pizza = fetch('/api/pizza').then(r => r.json());
// Ainda é Promise!

// ✅ CORRETO - Abres a caixa E guardas a pizza!
fetch('/api/pizza')
  .then(response => response.json()) // abrir caixa
  .then(pizza => this.pizza = pizza); // guardar pizza
```

🎯 Regra Mágica:

"FETCH dá caixa (Promise),
JSON abre caixa (Promise também!),
THEN finalmente dá o conteúdo!"

Ou visual:

fetch() → 📦 Promise<Response>

.json() → 📦 Promise<Data>

.then() → 🎁 Data!

💡 Async/Await (mais fácil):

javascript

// Async/await = Esperar pizza chegar

```
async function getPizza() {  
  const caixa = await fetch('/api/pizza'); // espera caixa  
  const pizza = await caixa.json(); // abre caixa  
  return pizza; // pizza!  
}
```

// "AWAIT = Aguardar Trabalho"

7. CSS LAYOUTS 📐

Mnemónica Flexbox: "FJAG" (Flex, Justify, Align, Gap)

F = Flex-direction → direção (Row ou Column)

J = Justify-content → eixo principal (horizontal se row)

A = Align-items → eixo cruzado (vertical se row)

G = Gap → espaço entre itens

🎨 Analogia Visual Flexbox:

CSS

```
/* 🌀 FLEXBOX = ORGANIZAR LIVROS */

.container {
  display: flex;

  /* 📖 DIRECTION = Prateleira horizontal/vertical? */
  flex-direction: row; /* ➡ horizontal (DEFAULT!) */
  flex-direction: column; /* ⬇ vertical */

  /* 📏 JUSTIFY = Espaçamento na prateleira */
  justify-content: space-between; /* 📖  📖  📖  */
  justify-content: center; /* 📖  📖  📖  */

  /* 📏 ALIGN = Altura dos livros */
  align-items: center; /* Todos no meio */
  align-items: flex-start; /* Todos em cima */

  /* 📏 GAP = Espaço entre livros */
  gap: 10px;
}
```

📝 Regra Nemónica:

"Justify é o Jogo horizontal,
Align é a Altura vertical!"

Ou:

"Row é o Rumo da fila,
Justify Junta como quiser!"

🌀 Grid vs Flexbox:

FLEXBOX = Fila de pessoas (1D - linha ou coluna)
GRID = Mesa com pratos (2D - linhas E colunas)

Use FLEXBOX quando: 1 direção

Use GRID quando: layout 2D complexo

Mnemónica Grid: "GTC-GAP" (Grid Template Columns - Gap)

CSS

```
.grid {  
  display: grid;  
  
  /* 📊 COLUMNS = Quantas colunas? */  
  grid-template-columns: 1fr 1fr 1fr; /* 3 iguais */  
  grid-template-columns: repeat(3, 1fr); /* mesmo */  
  
  /* 🎯 AUTO-FIT = Responsivo AUTOMÁTICO! */  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  /* "AUTO-fit Adapta Tudo Organizado" */  
  
  /* 📏 GAP = espaço */  
  gap: 20px;  
}
```

💡 Truque Responsivo Grid:

"Auto-fit Arranja Tudo,
Minmax Mede Mínimo e Máximo,
Grid fica Gigante ou Minúsculo!"

repeat(auto-fit, minmax(250px, 1fr))
👉 👉
automático pelo menos 250px

8. PRINCÍPIOS DE USABILIDADE 📖

Mnemónica: "LFR" (Learn, Flex, Robust)

L = LEARNABILITY → facilidade de aprender
F = FLEXIBILITY → flexibilidade/adaptação
R = ROBUSTNESS → robustez/resistência a erros

🎯 Sub-princípios (Mnemónica por inicial):

LEARNABILITY = "PSFGC"

P = Predictability → Prever o futuro
S = Synthesizability → Sintetizar passado
F = Familiarity → Familiar (conhecer)
G = Generalizability → Generalizar conhecimento
C = Consistency → Consistente sempre

FLEXIBILITY = "DTSM" + C (Dialogue, Task, Substitutivity, Multithreading + Customizability)

D = Dialogue initiative → Diálogo (quem inicia)
T = Task migratability → Tarefa (transferir)
S = Substitutivity → Substituir valores
M = Multithreading → Múltiplas tarefas
C = Customizability → Customizar

ROBUSTNESS = "ORRT" (como o Orcamento, Recursos, Resposta, Tarefa)

O = Observability → Observar estado ★
R = Recoverability → Recuperar de erros
R = Responsiveness → Resposta rápida
T = Task conformance → Tarefa suportada

⚠ ERRO CLÁSSICO:

"Observability NÃO é de Flexibility!
É de Robustness (ambos começam com O!)

OBSERVABILITY está na ROBUSTNESS
(Observar O Robusto sistema)"

🧠 Mapa Mental por Utilizador:

NOVATOS:

- Familiarity (reconhecer)
 - Predictability (prever)
 - Recoverability (recuperar erros)
- "Novato precisa de Norte (familiar),
Prever o caminho,
Recuperar quando erra!"

EXPERTS:

- Synthesizability (sintetizar info)
 - Customizability (personalizar)
 - Flexibility (flexível)
- "Expert é Sintético,
Customiza tudo,
Quer Flexibilidade!"

9. HEURÍSTICAS DE NIELSEN 🎯

Mnemónica: "VM-UCE-RFA-H" (10 heurísticas)

1. V = Visibility → Ver estado
2. M = Match → Mundo real
3. U = User control → Utilizador controla (undo)
4. C = Consistency → Consistência
5. E = Error prevention → Evitar erros
6. R = Recognition → Reconhecer (não memorizar)
7. F = Flexibility → Flexível (atalhos)
8. A = Aesthetic → Aparência minimalista
9. E = Error recovery → Erros claros (recuperar)
10. H = Help → Ajuda disponível

História Memorável:

"O VM (Virtual Machine) UCE (usa) RAF (real air force) para Help"

ou

"Ver Mundo, User Controla, Evita erros,
Reconhece Flexível Aparência,
Erros claros, Help disponível!"

Agrupamento Lógico:

VISIBILIDADE (saber o que está a acontecer):

1. Visibility of system status
9. Help recover from errors

PREVENÇÃO (evitar problemas):

5. Error prevention
6. Recognition rather than recall

CONTROLO (dar poder ao utilizador):

3. User control and freedom
7. Flexibility and efficiency

CONSISTÊNCIA (padrões):

2. Match real world
4. Consistency and standards
8. Aesthetic minimalist

AJUDA:

10. Help and documentation

Top 5 Mais Testadas:

1. 🚦 VISIBILITY - feedback do sistema
2. 🌐 MATCH - linguagem do utilizador
3. 🔄 USER CONTROL - undo/redo
4. 🚫 ERROR PREVENTION - prevenir > corrigir
5. 🧠 RECOGNITION - reconhecer > memorizar

10. MÉTODOS DE AVALIAÇÃO 🔍

Mnemónica: "AHCT-SN" (métodos principais)

ANALÍTICOS (sem utilizadores):

A = Avaliação Heurística

C = Cognitive Walkthrough

EMPÍRICOS (com utilizadores):

T = Think-aloud

S = SUS (questionário)

N = NASA TLX

🧠 Quando Usar (Decisão Rápida):

Sem utilizadores? → A ou C

Com utilizadores? → T, S ou N

Problemas gerais? → Avaliação Heurística

Learnability novatos? → Cognitive Walkthrough

Problemas específicos? → Think-aloud

Satisfação geral? → SUS

Comparar esforço? → NASA TLX

📝 Características:

AVALIAÇÃO HEURÍSTICA:

"3-5 Avaliadores Aham Heurísticas"

- 3-5 peritos
- Independentes
- Compilar duplicados
- 75% dos problemas

COGNITIVE WALKTHROUGH:

"Caminho Wobbly do Novato"

- Simular NOVATO (não expert!)
- Perguntas: vê? entende? associa? progresso?
- Foco: LEARNABILITY

THINK-ALLOUD:

"Think Alto enquanto usa"

- Verbalizar pensamentos
- Identificar problemas específicos
- Formativo

SUS:

"Score Usability Simplificado"

- 10 perguntas
- 0-100 pontos
- >68 = acima média

NASA TLX:

"NASA Taxa Load (carga) eXtrema"

- Carga mental
- COMPARAR interfaces
- Esforço/frustração

🎯 Decisão Rápida:

Preciso de...	Usar...	Tipo	
Rápido e barato	Aval. Heurística	Analít.	
Testar novatos	Cogn. Walkthrough	Analít.	
Ver utilizador	Think-aloud	Empír.	
Medir satisfação	SUS	Empír.	
Medir esforço	NASA TLX	Empír.	

11. WCAG (Acessibilidade Web) ♿

Mnemónica: "POUR" (em inglês, fácil!)

- P = PERCEIVABLE → Perceptível (ver/ouvir)
- O = OPERABLE → Operável (usar)
- U = UNDERSTANDABLE → Compreensível (entender)
- R = ROBUST → Robusto (compatível)

História:

"POUR water on accessibility"
(deitar água na acessibilidade)

ou em português:

"PODR conseguir usar o site"

P = Perceber (ver/ouvir)

O = Operar (teclado)

D = eNtenDer (compreender)

R = Rodar (funcionar em tudo)

Detalhes de cada princípio:

PERCEIVABLE:

- Alt text (imagens)

- Captions (vídeos)

- Contraste 4.5:1 (AA) ou 7:1 (AAA)

"Perceber Através de Contraste Alto"

OPERABLE:

- Todo via TECLADO

- Tempo suficiente

- Sem convulsões (blinking)

"Operável se Tudo por Teclado"

UNDERSTANDABLE:

- Linguagem clara

- Comportamento previsível

- Ajuda em formulários

"Understandable se Linguagem Clara"

ROBUST:

- HTML semântico

- Compatibilidade

- Tecnologias assistivas

"Robusto se HTML Semântico"

Níveis de Conformidade:

A = mínimo (básico)

AA = recomendado (4.5:1 contraste) ★

AAA = ideal (7:1 contraste)

"A é Aceitável,

AA é Adequado, ★ (este!)

AAA é Awesomely Accessible!"

12. MODELO DE NORMAN

Mnemônica: "EA" - Execução e Avaliação

E = EXECUÇÃO → Como Executar/Agir

A = AVALIAÇÃO → Avaliar Resultado

🍷 Analogia: Máquina de Café

FOSSO DA EXECUÇÃO:

"Não sei qual botão apertar!"

→ Affordances pouco claras

→ Controlos escondidos

→ Linguagem técnica

Minimizar com:

✓ Affordances claras (botões óbvios)

✓ Linguagem familiar ("Café" não "H2O quente + grãos")

✓ Mappings (posição botões = tipo café)

✓ Controlos visíveis

FOSSO DA AVALIAÇÃO:

"Apertei o botão... funcionou?"

→ Sem feedback

→ Sem indicador de progresso

→ Não sei se café está pronto

Minimizar com:

✓ Feedback imediato (som, luz)

✓ Indicadores (barra progresso)

✓ Confirmações visuais

✓ Estado claro (pronto/a fazer)

⚠ ERRO CLÁSSICO:

"Feedback minimiza fosso da EXECUÇÃO"

✗ ERRADO!

"Feedback minimiza fosso da AVALIAÇÃO"

✓ CORRETO!

Mnemónica:

"Feedback Fala do Fim (avaliação do resultado)

Affordances Ajudam no Agir (execução)"

ou

"AVALIAÇÃO precisa de fEEdback (mesmo som: é)

EXECUÇÃO precisa de Affordances (ambos começam com vogal)"

🎯 Princípios de Design:

A = Affordances → indica uso
M = Mappings → relação controlo↔efeito
C = Constraints → limita ações (podem ser +!)
F = Feedback → informa resultado

"AMCo o Feedback"

13. ACESSIBILIDADE

Mnemónica ARIA: "RLC" - Role, Label, Controls

R = Role → papel do elemento
L = Label → nome acessível
C = Controls → o que controla



Regra #1 do ARIA:

"HTML nativo PRIMEIRO, ARIA depois!"

<nav> ✓ (semântico)
<nav role="..."> ✗ (redundante)

<button> ✓ (semântico)
<div role="button"> ✗ (usar button!)

"ARIA é ARTificial,
HTML é Humano (natural)!"



Atributos Comuns:

role="button" → tipo de elemento
aria-label="Fechar" → nome quando sem texto visível
aria-expanded="false" → expandido/colapsado
aria-controls="menu" → controla elemento #menu
aria-hidden="true" → esconder de screen readers

"Role Labela Controls,
Expanded Hidden também!"



Screen Readers:

JAWS = Job Access (trabalho)
NVDA = NonVisual Desktop Access
VoiceOver = voz sobre (macOS/iOS)

"JAgar NVideojogos com Voz Over"

Mnemónica Tecnologias Assistivas:

S = Screen readers → lêem tela
M = screen Magnifiers → ampliam
V = Voice recognition → voz

"SMV ajuda a Ver"

14. JAVASCRIPT ⚡

addEventListener com `this`:

javascript

```
// ✗ Arrow function: this NÃO é o elemento  
element.addEventListener('click', (e) => {  
  this.classList.add('active'); // ✗ this errado!  
});
```

```
// ✓ Function normal: this É o elemento  
element.addEventListener('click', function(e) {  
  this.classList.add('active'); // ✓ this correto!  
});
```

Mnemónica:

"Arrow Aponta para outro **THIS**,
Function Fica no elemento **THIS**!"

ou

"Arrow **voa** (**this** escapa),
Function **fica** (**this** mantém)!"

Event Propagation:

javascript

```
// Bubbling = Bolhas sobem  
evento.stopPropagation(); // para a bolha
```

```
// Prevent default = Prevenir normal  
evento.preventDefault(); // previne comportamento
```

"Bolhas Borbulham para **cima** (bubbling),
Prevent Paralisa a **ação** (**default**)!"

🎯 TRUQUES FINAIS

Decisões Rápidas no Exame:

Feedback minimiza qual fosso?

→ AVALIAÇÃO (lembra: fEEdback tem É, Avaliação também)

Observability em qual princípio?

→ ROBUSTNESS (ambos começam com O!)

data: function() é Pinia?

→ NÃO! É Options API de components

reactive() funciona?

→ NÃO! reactive só objetos, ref para primitivos

:key="index" é boa prática?

→ SÓ se lista nunca muda/reordena

Props posso modificar?

→ NUNCA! São read-only (emitir evento)

fetch() retorna dados?

→ NÃO! Retorna Promise (precisa .then ou await)

watch executa imediatamente?

→ NÃO! (watchEffect sim)

Cleanup no onUnmounted?

→ SEMPRE! (senão memory leak)

<nav role="navigation">?

→ Redundante! (nav já é semântico)

REVISÃO RELÂMPAGO (5 minutos antes do teste)

1. PINIA: state/getters/actions (não data/methods)
2. REFS: ref() primitivos, reactive() objetos
3. TEMPLATE: :key único, v-model two-way
4. LIFECYCLE: onMounted fetch, onUnmounted cleanup
5. PROPS/EMITS: props ⬇ read-only, emits ⬆ eventos
6. FETCH: .then() duas vezes (response + json)
7. FOSSOS: Execução=agir, Avaliação=resultado
8. OBSERVABILITY: está em ROBUSTNESS!
9. MÉTODOS: Heurística=sem users, Think-aloud=com users
10. WCAG: POUR (Perceivable, Operable, Understandable, Robust)

BOA SORTE! 🍀

"Mnemónicas Memorizam Mais!"